COMP 018 DATABASE ADMINISTRATION • INTEGRATED MIDTERM PRACTICAL CHALLENGE

Midterm Challenge Comprehensive Technical Documentation

Peer Flaw Audit (Group 11), Incident Scenario Engineering & Owner Defense (Group 12)

Investigator: Group 07 (CampusSafe) | Audit Target: Group 11 (BoardTrack) | Assigned Designer: Group 12 (iBarangay) | Score Basis: 100 Points

Integrated Practical Challenge: Review, Design, Investigate, Troubleshoot, and Defend

Group Details & Project Directory

- **Course:** COMP 018 - Database Administration
- **Assessment:** Integrated Midterm Practical Challenge (100 Points)
- **Investigator / Owner Group: Group 07**
  - **System Name:** CampusSafe (Campus Hazard and Incident Reporting System)
  - **Repository:** `https://github.com/jeffersonorpesa2-ops/grp07-campussafe`
  - **Group Members:** Alba, Arreza, Orpesa, Robiso, Santos
- **Assigned Review Target (Designer Role): Group 11**
  - **System Name:** BoardTrack - Boarding House & Rental Management System
  - **Target Repository:** `https://github.com/karlrobertc-art/BoardTrack`
  - **Group 11 Members:** Barcarse, Adrian; Cataluña, Zyruz Gabriel; Camu, Edgar; Policarpio, Kurt; Crucillo, Karl Robert
- **Assigned Designer for Our System (CampusSafe): Group 12**
  - **System Name:** iBarangay (Barangay Document Request System)
  - **Group 12 Members:** Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix

Table of Contents

1. Executive Summary & System Links
2. Class Tracker Entries (Midterm Tab Rows 8 & 11)
3. Part A: Flaw Review - Group 11 (BoardTrack)
4. Part B: Incident Scenario Design - Group 11 (BoardTrack)
   - Incident 1: G11-INC1 (Cyclic Room Swap Trigger Deadlock)
   - Incident 2: G11-INC2 (Uncommitted Row Lock & Lost Update Anomaly)
   - Incident 3: G11-INC3 (Utility Bill Date Check Constraint Violation)
   - Incident 4: G11-INC4 (Role Privilege Contradiction - Missing DELETE)
   - Incident 5: G11-INC5 (Non-Transactional Seeding Lockout)
5. Executable Fault & Reset Scripts for Group 11 (Answer Key)
6. Part C & D: Owner Investigation & Troubleshooting Plan (CampusSafe - Group 12 Incidents)
   - Incident 1: G07-INC1 (Excessive UPDATE Privileges on Sensitive Columns)
   - Incident 2: G07-INC2 (Cascading Deletions Purging Audit & Compliance History)
   - Incident 3: G07-INC3 (Missing Database Trigger for Status History Auditing)
   - Incident 4: G07-INC4 (Missing Composite Unique Constraint on Physical Locations)
   - Incident 5: G07-INC5 (Missing Conditional Priority Requirement on Verified Reports)
7. Part E: Designer Review Scorecard (Evaluating Group 11)
8. Feedback Sent to Project Owners (Group 11)
9. Contribution Record
10. AI Usage Declaration

1. Executive Summary & System Links

1.1 Scope and Objective

This midterm assessment executes an integrated peer audit and defense model under the principles of production database administration. In accordance with the course rubric, Group 07 has executed two primary duties:

1. **Designer Role (Auditing Group 11 - BoardTrack):** Performed static codebase analysis, schema integrity auditing, concurrency stress testing, and RBAC privilege verification on BoardTrack. Identified 10 evidence-based flaws and engineered 5 realistic, safe, and recoverable incident scenarios covering concurrency deadlocks, row lock contention, check constraint rejections, role privilege omissions, and non-atomic bootstrap seeding.
2. **Owner Role (Defending Group 07 - CampusSafe):** Investigated, analyzed, and remediated the 5 incident scenarios designed by Group 12 for CampusSafe. Applied an 8-step DBA troubleshooting method (*Observe -> Verify -> Hypothesize -> Test -> Act -> Verify -> Document -> Prevent*) on an isolated test environment on PostgreSQL 18.

1.2 Repository & Class Tracker Links

- **CampusSafe Project Repository:** `https://github.com/jeffersonorpesa2-ops/grp07-campussafe`
- **CampusSafe User Roles Documentation:** `https://github.com/jeffersonorpesa2-ops/grp07-campussafe/blob/main/docs/user-roles.md`
- **BoardTrack Audited Repository:** `https://github.com/karlrobertc-art/BoardTrack`
- **Class Google Sheets Project Tracker:** `[BSIT 3-1] - PROJECT TRACKER.xlsx` (Midterm Tab, Rows 8 & 11)

2. Class Tracker Entries (Midterm Tab Rows 8 & 11)

2.1 Group 07 as Owner (Row 8 in Midterm Tab - Investigating Group 12's Incidents)

INCIDENT ID	PROJECT (OWNER)	DESIGNER GROUP	FLAW FOUND	OWNER'S RESPONSE TO SAID FLAW	INCIDENT (SYMPTOMS ONLY)	OWNER'S FINDING
G07-INC1	G07	G12	The application runtime database role has excessive UPDATE privileges on the app_user table, allowing modifications to sensitive user information, including assigned roles and account statuses.	Confirmed. Runtime role was granted full table-wide UPDATE without column-level restrictions.	An application account modifies a user's assigned role or account status even though the operation should be restricted to authorized administrators.	Runtime role had table-wide UPDATE granted on app_user. Fixed by revoking table-wide UPDATE and granting column-level UPDATE only on non-sensitive columns (full_name, email, password_hash).
G07-INC2	G07	G12	The database schema uses ON DELETE CASCADE on the foreign key relationships between hazard_report and its dependent tables, including report_status_history, maintenance_action, assignment, and attachment. Deleting a parent report automatically removes its associated historical records and supporting information.	Confirmed. Cascading deletes violate compliance retention requirements for audited hazard reports.	An administrator deletes an old hazard report, causing its associated status history, maintenance records, assignments, and uploaded evidence to disappear from the database.	Foreign keys on dependent audit/history tables configured with ON DELETE CASCADE. Fixed by altering foreign keys to ON DELETE RESTRICT and implementing soft deletion (is_deleted/deleted_at flag).
G07-INC3	G07	G12	The database does not implement automatic triggers to record changes made to hazard report statuses. Instead, the system relies on application-level operations to insert records into the report_status_history table.	Confirmed. Relying solely on PHP controllers created a blind spot for direct SQL changes and batch updates.	An administrator directly changes a hazard report's status through a database management tool, but the corresponding status change does not appear in the report's historical records.	Status history tracking was handled at application level without an underlying database trigger. Fixed by creating an AFTER UPDATE trigger on hazard_report that automatically logs status transitions to report_status_history.
G07-INC4	G07	G12	The campus_location table does not implement a unique constraint for the combination of building, floor, and room or area, allowing duplicate location entries to be stored in the database.	Confirmed. Schema relied solely on surrogate PK location_id and omitted business candidate key constraints.	Administrators encounter multiple entries representing the same building, floor, and room when selecting locations for hazard reports.	Table campus_location lacked a composite UNIQUE constraint on (building, floor, room). Fixed by de-duplicating records, re-pointing hazard reports, and adding UNIQUE (building, floor, room).
G07-INC5	G07	G12	The database allows hazard reports to reach the Verified status without an assigned priority because the priority column permits NULL values and does not enforce a requirement for verified reports.	Confirmed. Priority remained nullable across all states without conditional state validation.	Maintenance personnel encounter verified hazard reports without priority classifications, making it difficult to determine which incidents require immediate attention.	Table hazard_report permitted status = 'VERIFIED' with NULL priority. Fixed by backfilling verified records with 'MEDIUM' and adding CHECK (status <> 'VERIFIED' OR priority IS NOT NULL).

3. Part A: Flaw Review - Group 11 (BoardTrack)

The following evidence-based flaws were identified during local testing, schema review, and code inspection of `karlrobertc-art/BoardTrack`.

Flaw ID	Area & Activity	Component / File Location	Flaw Description & Evidence	Severity	Recommended Fix

4. Part B: Incident Scenario Design - Group 11 (BoardTrack)

Incident 1: G11-INC1 (Cyclic Room Swap Trigger Deadlock)

- **Incident ID:** `G11-INC1` (Linked Flaw: `G11-FL01`)
- **Variant & Activity Area:** Trigger Concurrency / Deadlock Contention (Activity 4 & 5)
- **Difficulty:** Hard (Estimated Time: 35 Minutes)

Public Tracker Brief (Symptoms Only)

> ***Symptoms Reported:*** *The landlord and caretaker simultaneously attempted to reassign two tenants between Room 101 and Room 102 through the Leases management module. During submission, both browser tabs hung for several seconds. One user's browser crashed with an unhandled server error, while the other completed, leaving room availability inconsistent.*

Public Evidence Available for Investigation

- Application terminal error log: `error: deadlock detected (SQLSTATE 40P01)`
- PostgreSQL server logs showing conflicting PID wait queues on relation `room`.

Private Instructor Answer Key

- **Root Cause:** In `001_schema.sql`, trigger `sync_room_status()` updates room rows sequentially: `NEW.room_id` followed by `OLD.room_id`. Swapping leases creates an inverted lock order (AB-BA deadlock).

- **Corrective Action:** Rewrite trigger update query to update both rooms atomically in sorted primary key order or via `WHERE room_id IN (OLD.room_id, NEW.room_id)`.
- **Verification:** Run `SELECT room_id, status FROM room WHERE room_id IN (1, 2);` and re-run concurrent swap script to confirm zero deadlocks.

Incident 2: G11-INC2 (Uncommitted Row Lock & Lost Update Anomaly)

- **Incident ID:** `G11-INC2` (Linked Flaws: `G11-FL02`, `G11-FL03`)
- **Variant & Activity Area:** Concurrency / Lock Blocking & Missing OCC (Activity 5)
- **Difficulty:** Medium (Estimated Time: 25 Minutes)

Public Tracker Brief (Symptoms Only)

> ***Symptoms Reported:*** *Caretaker attempts to update the rental rate for Room 101, but the save button spins and the browser tab hangs indefinitely without completing. Minutes later, changes saved earlier by the landlord on another room spontaneously disappeared.*

Public Evidence Available for Investigation

- Active database lock catalog (`pg_locks` and `pg_stat_activity`).
- Process query state showing `waiting = true` and `state = 'idle in transaction'`.

Private Instructor Answer Key

- **Root Cause:** A client session initiated a transaction modifying Room 1 (`UPDATE room SET monthly_rate = ...`) without issuing `COMMIT` or `ROLLBACK`, holding an exclusive tuple lock. Additionally, room update routes lack version tracking, causing stale overwrites.
- **Corrective Action:** Identify the blocking process via `pg_stat_activity` and execute `SELECT pg_terminate_backend(<blocking_pid>);`. Add a `version` column and check `WHERE room_id = $1 AND version = $2`.
- **Verification:** Confirm no blocking processes exist using `pg_stat_activity` and verify that concurrent updates succeed or reject stale forms gracefully.

Incident 3: G11-INC3 (Utility Bill Date Check Constraint Violation)

- **Incident ID:** `G11-INC3` (Linked Flaw: `G11-FL08`)
- **Variant & Activity Area:** Schema Constraints / Validation (Activity 3 & 5)
- **Difficulty:** Easy (Estimated Time: 15 Minutes)

Public Tracker Brief (Symptoms Only)

> ***Symptoms Reported:*** *The caretaker attempted to record an electric utility bill for Sunshine Boarding House using today's date in the date picker. The submission failed with an alert: *"A value is outside the allowed range (check amounts, dates, status)."* No bill was created.*

Public Evidence Available for Investigation

- Application console log: `UnhandledPromiseRejection: error: new row for relation "utility_bill" violates check constraint "utility_bill_billing_period_check"`
- Target SQLSTATE: `23514` (`check_violation`).

Private Instructor Answer Key

- **Root Cause:** Table `utility_bill` contains constraint `CHECK (EXTRACT(DAY FROM billing_period) = 1)`. Web form sends user-selected date directly (e.g., `2026-11-07`), violating the Day 1 restriction.
- **Corrective Action:** Modify `src/routes/bills.js` to normalize the date parameter to the first day of the month before executing the `INSERT` query.
- **Verification:** Execute `SELECT COUNT(*) FROM utility_bill WHERE billing_period = '2026-11-01';` and submit a bill through the web UI using any day of the month.

Incident 4: G11-INC4 (Role Privilege Contradiction - Missing DELETE)

- **Incident ID:** `G11-INC4` (Linked Flaw: `G11-FL09`)
- **Variant & Activity Area:** Database Roles & Least-Privilege Access Control (Activity 4)
- **Difficulty:** Medium (Estimated Time: 20 Minutes)

Public Tracker Brief (Symptoms Only)

> ***Symptoms Reported:*** *When testing the application under the documented least-privilege database role (`boardtrack_app`), the landlord user is unable to delete obsolete payment records or test leases. The browser returns an HTTP 500 error page.*

Public Evidence Available for Investigation

- PostgreSQL engine log: `ERROR: permission denied for table payment at character 13 (SQLSTATE 42501)`
- Catalog query on `information_schema.role_table_grants`.

Private Instructor Answer Key

- **Root Cause:** In `docs/user-roles.md`, the DBA role setup plan explicitly grants `SELECT, INSERT, UPDATE` to `app_runtime_grp`, omitting `DELETE`. However, the web application exposes delete endpoints.
- **Corrective Action:** Grant the missing delete privilege to the runtime group: `GRANT DELETE ON ALL TABLES IN SCHEMA public TO app_runtime_grp;`.
- **Verification:** Run `DELETE FROM payment WHERE payment_id = 999;` as user `boardtrack_app` to verify success, and verify that `app_readonly_grp` is still denied.

Incident 5: G11-INC5 (Non-Transactional Seeding Lockout)

- **Incident ID:** `G11-INC5` (Linked Flaw: `G11-FL05`)
- **Variant & Activity Area:** Bootstrap Automation, Seeding & Recovery (Activity 3 & 6)
- **Difficulty:** Easy (Estimated Time: 15 Minutes)

Public Tracker Brief (Symptoms Only)

> ***Symptoms Reported:*** *During a fresh system deployment with custom sample data, a syntax error in the seed file halted initial setup. After fixing the seed file and restarting the server, the application started but reported "seed skipped", leaving the system with missing sample rooms and zero user login accounts.*

Public Evidence Available for Investigation

- Server startup log: `[bootstrap] property exists, skipping seed`
- Database state: `SELECT COUNT(*) FROM property;` returns `> 0`, but `SELECT COUNT(*) FROM app_user;` returns `0`.

Private Instructor Answer Key

- **Root Cause:** In `src/bootstrap.js`, function `seed()` executes `001_sample_data.sql` in autocommit mode. If an error occurs midway, preceding `INSERT INTO property` statements commit. On restart, the check `(property count + app_user count) > 0` evaluates to true, permanently bypassing user creation.
- **Corrective Action:** Wrap `bootstrap.seed()` in an explicit client transaction (`BEGIN ... COMMIT`) with automatic rollback on error.
- **Verification:** Drop and recreate database, trigger a simulated mid-seed failure, and confirm that all tables rollback cleanly to 0 rows.

---

5. Executable Fault & Reset Scripts for Group 11 (Answer Key)

5.1 G11-INC1 (Deadlock)

- **Fault (`fault_inc01_deadlock.sql`):**

```
BEGIN;
UPDATE room SET status = 'AVAILABLE' WHERE room_id = 1;
SELECT pg_sleep(2);
UPDATE room SET status = 'OCCUPIED' WHERE room_id = 2;
COMMIT;
```

- **Reset (`reset_inc01_deadlock.sql`):**

```
UPDATE room SET status = 'AVAILABLE' WHERE room_id IN (1, 2);
```

5.2 G11-INC2 (Lock Contention)

- **Fault (`fault_inc02_lock_blocking.sql`):**

```
BEGIN;
UPDATE room SET monthly_rate = monthly_rate + 250 WHERE room_id = 1;
-- Keep uncommitted
```

- **Reset (`reset_inc02_lock_blocking.sql`):**

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE query LIKE '%UPDATE room SET monthly_rate%' AND pid <> pg_backend_pid();
```

5.3 G11-INC3 (Check Constraint)

- **Fault (`fault_inc03_billing_constraint.sql`):**

```
INSERT INTO utility_bill (lease_id, utility_type, billing_period, amount, status)
VALUES (1, 'ELECTRICITY', '2026-11-07', 1450.00, 'UNPAID');
```

- **Reset (`reset_inc03_billing_constraint.sql`):**

```
DELETE FROM utility_bill WHERE billing_period = '2026-11-07';
```

5.4 G11-INC4 (Role Privilege)

- **Fault (`fault_inc04_role_permission.sql`):**

```
REVOKE DELETE ON ALL TABLES IN SCHEMA public FROM app_runtime_grp;
```

- **Reset (`reset_inc04_role_permission.sql`):**

```
GRANT DELETE ON ALL TABLES IN SCHEMA public TO app_runtime_grp;
```

5.5 G11-INC5 (Seed Recovery)

- **Fault (`fault_inc05_seed_corruption.sql`):**

```
INSERT INTO property (property_name, address, property_type)
VALUES ('Corrupted Partial Seed House', '123 Ghost Lane', 'APARTMENT');
DELETE FROM app_user;
```

- **Reset (`reset_inc05_seed_corruption.sql`):**

```
DELETE FROM property WHERE property_name = 'Corrupted Partial Seed House';
```

---

6. Part C & D: Owner Investigation & Troubleshooting Plan (CampusSafe - Group 12 Incidents)

Group 07 executed the disciplined 8-step DBA troubleshooting method against the 5 incidents submitted by Group 12 on a dedicated disposable database (`campussafe_midterm_test`).

Incident 1: G07-INC1 (Excessive UPDATE Privileges on Sensitive Columns)

- **Incident ID:** `G07-INC1` (Designer: Group 12)
- **Variant & Activity Area:** Role Privileges & Least-Privilege Access Control (Activity 4)

1. Observe (Symptoms Reported)

> *An application account modifies a user's assigned role or account status even though the operation should be restricted to authorized administrators.*

2. Verify (Database Inspection)

Inspection of `information_schema.table_privileges` revealed that the runtime database role `campussafe_app` (member of `app_runtime_grp`) held table-level `UPDATE` privileges on `app_user`:

```
SELECT grantee, table_name, privilege_type
FROM information_schema.table_privileges
WHERE table_name = 'app_user' AND grantee = 'app_runtime_grp';
```

Output: `grantee: app_runtime_grp | privilege_type: UPDATE`.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** The Activity 4 database setup script issued `GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_runtime_grp;`, indiscriminately granting column-level write access to sensitive administrative columns (`role`, `status`) on

`app_user`.

- **Hypothesis B (Ruled Out):** Application authorization flaw alone allowed role updates. Ruled out because direct database query execution under `campussafe_app` succeeded, proving the database engine failed to enforce least-privilege defense-in-depth.

4. Troubleshooting Plan & Corrective Action

- **Troubleshooting Plan:** Revoke broad table-wide UPDATE privilege from `app_runtime_grp` on `app_user` and replace with granular column-level UPDATE grants on safe user-profile attributes only.
- **Account Used:** `postgres` (DBA administrative maintenance).
- **Corrective SQL Action:**

```
-- Step 1: Revoke full-table update
REVOKE UPDATE ON app_user FROM app_runtime_grp;

-- Step 2: Grant column-level UPDATE on safe fields only
GRANT UPDATE (full_name, email, password_hash, updated_at) ON app_user TO app_runtime_grp;
```

- **Rollback Plan:** `GRANT UPDATE ON app_user TO app_runtime_grp;`

5. Verification

- **Negative Test (Must Fail):** Executed under `campussafe_app`:

```
SET ROLE campussafe_app;
UPDATE app_user SET role = 'admin' WHERE user_id = 2;
```

Result: `ERROR: permission denied for column "role" of relation "app_user" (SQLSTATE 42501)` -> **PASS**.

- **Positive Test (Must Succeed):** Executed under `campussafe_app`:

```
UPDATE app_user SET full_name = 'Updated User Name' WHERE user_id = 2;
```

Result: `UPDATE 1` -> **PASS**.

6. Prevention Recommendation

Establish automated privilege inspection queries in CI/CD pipeline that flag any table-wide `UPDATE` or `DELETE` grants on authentication and role catalogs (`app_user`, `roles`).

Incident 2: G07-INC2 (Cascading Deletions Purging Audit & Compliance History)

- **Incident ID:** `G07-INC2` (Designer: Group 12)
- **Variant & Activity Area:** Schema Design, Referential Integrity & Audit Retention (Activity 3 & 4)

1. Observe (Symptoms Reported)

> *An administrator deletes an old hazard report, causing its associated status history, maintenance records, assignments, and uploaded evidence to disappear from the database.*

2. Verify (Database Inspection)

Inspection of foreign key metadata via `pg_constraint` showed `confdeltype = 'c'` (`c` = CASCADE) on all child relations referencing `hazard_report`:

```
SELECT conname, conrelid::regclass, confdeltype
FROM pg_constraint
WHERE confrelid = 'hazard_report'::regclass;
```

Output: Foreign keys `fk_history_report`, `fk_maintenance_report`, `fk_assignment_report`, and `fk_attachment_report` all evaluated to `confdeltype: c`.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** Schema migration scripts configured `ON DELETE CASCADE` for convenience during early development, violating regulatory compliance and audit trail preservation rules.
- **Hypothesis B (Ruled Out):** Application trigger deleted child rows explicitly. Ruled out by verifying that no `BEFORE/AFTER DELETE` triggers existed on `hazard_report`.

4. Troubleshooting Plan & Corrective Action

- **Troubleshooting Plan:** Alter all child table foreign keys to `ON DELETE RESTRICT` (blocking hard deletion while dependent audit records exist) and add a soft-delete mechanism (`is_deleted`, `deleted_at`) to `hazard_report`.
- **Corrective SQL Action:**

```
-- Step 1: Restrict deletions on dependent audit tables
ALTER TABLE report_status_history DROP CONSTRAINT IF EXISTS fk_status_history_report;
ALTER TABLE report_status_history ADD CONSTRAINT fk_status_history_report
  FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE maintenance_action DROP CONSTRAINT IF EXISTS fk_maintenance_report;
ALTER TABLE maintenance_action ADD CONSTRAINT fk_maintenance_report
  FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE assignment DROP CONSTRAINT IF EXISTS fk_assignment_report;
ALTER TABLE assignment ADD CONSTRAINT fk_assignment_report
  FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

ALTER TABLE attachment DROP CONSTRAINT IF EXISTS fk_attachment_report;
ALTER TABLE attachment ADD CONSTRAINT fk_attachment_report
  FOREIGN KEY (report_id) REFERENCES hazard_report(report_id) ON DELETE RESTRICT;

-- Step 2: Implement soft deletion on hazard_report
ALTER TABLE hazard_report ADD COLUMN IF NOT EXISTS is_deleted BOOLEAN DEFAULT FALSE;
ALTER TABLE hazard_report ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ;
```

- **Rollback Plan:** Re-add foreign keys with `ON DELETE CASCADE`.

5. Verification

- **Negative Test (Must Fail):** Executed under DBA role:

```
DELETE FROM hazard_report WHERE report_id = 1;
```

Result: `ERROR: update or delete on table "hazard_report" violates foreign key constraint "fk_status_history_report" on table "report_status_history" (SQLSTATE 23503)` -> **PASS**.

- **Positive Test (Must Succeed):**

```
UPDATE hazard_report SET is_deleted = TRUE, deleted_at = NOW() WHERE report_id = 1;
SELECT is_deleted, deleted_at FROM hazard_report WHERE report_id = 1;
```

Result: Report marked deleted while 100% of status history, attachments, and assignments remain intact -> **PASS**.

6. Prevention Recommendation

Establish an explicit DBA policy prohibiting `CASCADE` delete rules on any compliance, incident, or financial audit records; mandate soft-deletion patterns.

Incident 3: G07-INC3 (Missing Database Trigger for Status History Auditing)

- **Incident ID:** `G07-INC3` (Designer: Group 12)
- **Variant & Activity Area:** Database Triggers & Automatic Auditing (Activity 3 & 5)

1. Observe (Symptoms Reported)

> *An administrator directly changes a hazard report's status through a database management tool, but the corresponding status change does not appear in the report's historical records.*

2. Verify (Database Inspection)

Inspection of catalog `pg_trigger` confirmed that no status change triggers were attached to table `hazard_report`:

```
SELECT tgname, tgtype, tgenabled
FROM pg_trigger
WHERE tgrelid = 'hazard_report'::regclass AND NOT tgisinternal;
```

Output: `(0 rows)` -> Status logging was solely handled by application PHP controller logic.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** State transitions relied on application-level `INSERT` statements inside web controllers rather than an underlying database trigger. Manual updates, batch migrations, or direct queries bypassed history logging.
- **Hypothesis B (Ruled Out):** Trigger was installed but disabled. Ruled out as no trigger definition existed in the database catalog.

4. Troubleshooting Plan & Corrective Action

- **Troubleshooting Plan:** Implement an `AFTER UPDATE OF status` trigger on `hazard_report` that automatically writes the previous status, new status, timestamp, and changing user into `report_status_history`.
- **Corrective SQL Action:**

```
CREATE OR REPLACE FUNCTION trg_log_hazard_status_change()
RETURNS TRIGGER AS $$
BEGIN
  IF (OLD.status IS DISTINCT FROM NEW.status) THEN
    INSERT INTO report_status_history (
      report_id,
      previous_status,
      new_status,
      changed_by,
      change_timestamp,
      remarks
    ) VALUES (
      NEW.report_id,
      OLD.status,
      NEW.status,
      NULL,
      NOW(),
      'Database trigger audit log'
    );
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_hazard_report_status_audit ON hazard_report;
CREATE TRIGGER trg_hazard_report_status_audit
AFTER UPDATE OF status ON hazard_report
FOR EACH ROW
EXECUTE FUNCTION trg_log_hazard_status_change();
```

- **Rollback Plan:** `DROP TRIGGER trg_hazard_report_status_audit ON hazard_report; DROP FUNCTION trg_log_hazard_status_change();`

5. Verification

- **Test:** Executed direct SQL update via psql:

```
UPDATE hazard_report SET status = 'IN_PROGRESS' WHERE report_id = 1;
SELECT report_id, previous_status, new_status, change_timestamp, remarks
FROM report_status_history
WHERE report_id = 1
ORDER BY change_timestamp DESC LIMIT 1;
```

Result: New audit row automatically populated showing transition from `PENDING` to `IN_PROGRESS` -> **PASS**.

6. Prevention Recommendation

Enforce all core audit trails and compliance state changes at the database engine level via triggers so that external tools or API integrations cannot bypass audit logging.

Incident 4: G07-INC4 (Missing Composite Unique Constraint on Physical Locations)

- **Incident ID:** `G07-INC4` (Designer: Group 12)
- **Variant & Activity Area:** Schema Constraints & Normalization (Activity 3 & 5)

1. Observe (Symptoms Reported)

> *Administrators encounter multiple entries representing the same building, floor, and room when selecting locations for hazard reports.*

2. Verify (Database Inspection)

Inspection of `campus_location` constraints confirmed the table possessed only a surrogate primary key `location_id` and zero composite uniqueness constraints on location dimensions:

```
SELECT conname, contype, pg_get_constraintdef(oid)
FROM pg_constraint
WHERE conrelid = 'campus_location'::regclass;
```

Output: Only `pk_campus_location (PRIMARY KEY (location_id))` was present. Querying duplicates:

```
SELECT building, floor, room, COUNT(*)
FROM campus_location
GROUP BY building, floor, room HAVING COUNT(*) > 1;
```

Result: Revealed duplicate location entries stored with distinct IDs.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** Schema design incorporated surrogate primary key `location_id` but omitted the corresponding candidate key constraint on `(building, floor, room)`.
- **Hypothesis B (Ruled Out):** Application dropdown bug generated duplicate visual items. Ruled out as duplicate rows physically existed in the database table.

4. Troubleshooting Plan & Corrective Action

- **Troubleshooting Plan:** Cleanse existing duplicate rows by updating child references in `hazard_report` to the lowest primary `location_id`, purge duplicate location rows, and add a composite `UNIQUE` constraint.
- **Corrective SQL Action:**

```
-- Step 1: Re-point foreign keys to primary location ID
WITH ranked_locations AS (
  SELECT location_id,
         FIRST_VALUE(location_id) OVER (PARTITION BY building, floor, room ORDER BY location_id) as keep_id,
         ROW_NUMBER() OVER (PARTITION BY building, floor, room ORDER BY location_id) as rn
  FROM campus_location
)
UPDATE hazard_report hr
SET location_id = rl.keep_id
FROM ranked_locations rl
WHERE hr.location_id = rl.location_id AND rl.rn > 1;

-- Step 2: Delete duplicate location rows
WITH duplicates AS (
  SELECT location_id,
         ROW_NUMBER() OVER (PARTITION BY building, floor, room ORDER BY location_id) as rn
  FROM campus_location
)
DELETE FROM campus_location
WHERE location_id IN (SELECT location_id FROM duplicates WHERE rn > 1);

-- Step 3: Add composite UNIQUE constraint
ALTER TABLE campus_location
ADD CONSTRAINT uq_campus_location_bldg_floor_room
UNIQUE (building, floor, room);
```

- **Rollback Plan:** `ALTER TABLE campus_location DROP CONSTRAINT uq_campus_location_bldg_floor_room;`

5. Verification

- **Negative Test (Must Fail):** Executed duplicate insertion:

```
INSERT INTO campus_location (building, floor, room) VALUES ('Main Building', '2nd Floor', 'Room 201');
INSERT INTO campus_location (building, floor, room) VALUES ('Main Building', '2nd Floor', 'Room 201');
```

Result: `ERROR: duplicate key value violates unique constraint "uq_campus_location_bldg_floor_room" (SQLSTATE 23505)` -> **PASS**.

6. Prevention Recommendation

Mandate composite natural keys on all dimensional lookup tables to prevent physical record redundancy.

Incident 5: G07-INC5 (Missing Conditional Priority Requirement on Verified Reports)

- **Incident ID:** `G07-INC5` (Designer: Group 12)
- **Variant & Activity Area:** Schema Integrity, Check Constraints & Data Quality (Activity 3 & 5)

1. Observe (Symptoms Reported)

> *Maintenance personnel encounter verified hazard reports without priority classifications, making it difficult to determine which incidents require immediate attention.*

2. Verify (Database Inspection)

Inspection of `hazard_report` column definitions confirmed that `priority` was defined as `VARCHAR(20) NULL` with no conditional check constraints enforcing priority values upon verification:

```
SELECT column_name, is_nullable, data_type
FROM information_schema.columns
WHERE table_name = 'hazard_report' AND column_name = 'priority';
```

Output: `is_nullable: YES`. Querying verified reports with missing priority:

```
SELECT COUNT(*) FROM hazard_report WHERE status = 'VERIFIED' AND priority IS NULL;
```

Result: Returns non-zero count of unclassified verified reports.

3. Hypothesize (Root Cause Reasoning)

- **Hypothesis A (Confirmed):** Priority was configured as nullable because initial submissions start as `PENDING` without a triage rating. However, the database lacked a conditional check constraint requiring priority once status transitions to `VERIFIED`.
- **Hypothesis B (Ruled Out):** Maintenance view filtered priority incorrectly. Ruled out as the underlying table held NULL priority data.

4. Troubleshooting Plan & Corrective Action

- **Troubleshooting Plan:** Backfill existing verified reports holding NULL priority with default `"MEDIUM"`, then add a conditional `CHECK` constraint requiring non-null priority for verified reports.
- **Corrective SQL Action:**

```
-- Step 1: Backfill existing NULLs
UPDATE hazard_report
SET priority = 'MEDIUM'
WHERE status = 'VERIFIED' AND priority IS NULL;

-- Step 2: Add conditional CHECK constraint
ALTER TABLE hazard_report
ADD CONSTRAINT chk_verified_report_requires_priority
CHECK (status <> 'VERIFIED' OR priority IS NOT NULL);
```

- **Rollback Plan:** `ALTER TABLE hazard_report DROP CONSTRAINT chk_verified_report_requires_priority;`

5. Verification

- **Negative Test (Must Fail):** Executed invalid update:

```
UPDATE hazard_report SET status = 'VERIFIED', priority = NULL WHERE report_id = 1;
```

Result: `ERROR: new row for relation "hazard_report" violates check constraint "chk_verified_report_requires_priority" (SQLSTATE 23514)` -> **PASS**.

- **Positive Test (Must Succeed):**

```
UPDATE hazard_report SET status = 'VERIFIED', priority = 'HIGH' WHERE report_id = 1;
```

Result: `UPDATE 1` -> **PASS**.

6. Prevention Recommendation

Incorporate state-machine conditional constraints (`CHECK (status != 'X' OR field IS NOT NULL)`) for all lifecycle entities to enforce progressive data completeness.

---

7. Part E: Designer Review Scorecard (Evaluating Group 11)

***[NOTE]** *Status: Awaiting Final Troubleshooting Submission from Group 11.* Once Group 11 submits their investigation report on our 5 designed incidents, Group 07 will evaluate their results against our Answer Key below.*

Incident ID	Target Flaw	Owner Found Cause?	Corrective Action Valid?	Score (1-5)	Designer Notes & Feedback
G11-INC1	Trigger cyclic row lock deadlock	*[Pending]*	*[Pending]*	/5	Evaluates if trigger order inversion was identified.
G11-INC2	Uncommitted row lock & missing OCC	*[Pending]*	*[Pending]*	/5	Evaluates if blocked PID was isolated in pg_stat_activity.
G11-INC3	Billing period check constraint violation	*[Pending]*	*[Pending]*	/5	Evaluates date normalization in route handler.
G11-INC4	Missing DELETE in runtime role grants	*[Pending]*	*[Pending]*	/5	Evaluates detection of role grant contradiction.
G11-INC5	Non-transactional seeding lockout	*[Pending]*	*[Pending]*	/5	Evaluates client transaction encapsulation in bootstrap.js.

8. Feedback Sent to Project Owners (Group 11)

To: Group 11 (BoardTrack Project Team) **From:** Group 07 (CampusSafe Quality & Database Review Team)

> ***Summary of Audit Findings:*** *BoardTrack exhibits a clean project architecture, responsive UI components, and strong relational schema definitions with appropriate exclusion constraints. However, our database administration stress testing identified critical production risks: 1.* ***Trigger Concurrency:*** *Sequential updates in `sync_room_status` are vulnerable to row-level cyclic deadlocks during peak concurrent reassignments. 2.* ***Transaction Scoping:*** *Bootstrap seeding and database migrations require full client-side transaction wrapping (`BEGIN...COMMIT`) to prevent permanent partial-state corruption. 3.* ***Role Specification Contradiction:*** *The least-privilege plan in `docs/user-roles.md` omits `DELETE` permissions, breaking application landlord functionality when runtime role hardening is activated. 4.* ***Input Normalization:*** *Form handlers should sanitize and truncate billing dates to the first day of the month prior to insertion to prevent avoidable check constraint crashes.*

9. Contribution Record

Group Member	Designer Role Responsibilities (on Group 11)	Owner / Investigator Responsibilities (on CampusSafe)
Alba	Analyzed role permissions in docs/user-roles.md vs route endpoints; drafted G11-INC4 fault script.	Investigated G07-INC1 (role privileges) and executed column-level grant remediations.
Arreza	Inspected schema constraints, date checks, and billing routes; designed G11-INC3 scenario.	Investigated G07-INC4 & G07-INC5; implemented composite unique and conditional check constraints.
Orpesa	Executed concurrency and lock analysis; designed cyclic deadlock scenario (G11-INC1).	Coordinated troubleshooting workflows, investigated G07-INC3 (audit triggers), and authored trigger functions.
Robiso	Audited bootstrap seeding and migration atomicity; designed recovery scenario (G11-INC5).	Investigated G07-INC2 (cascading deletes), re-engineered foreign keys to RESTRICT, and added soft deletion.
Santos	Formatted Flaw Register, coordinated public tracker entries, and documented Answer Key.	Compiled evidence logs, verified negative permission tests, and formatted final report deliverables.

10. AI Usage Declaration

- **Tool Utilized:** Google DeepMind / Antigravity (Gemini 3.8 Flash)
- **Tasks Delegated:** Codebase static analysis assistance, deadlock sequence validation, SQL script template drafting, and document structure formatting according to the course rubric.
- **Independent Verification Method:** Every identified SQLSTATE error code (`40P01`, `23514`, `42501`, `57014`, `23503`, `23505`) and catalog query was independently verified against a live local PostgreSQL 18 instance. All route contradictions were cross-referenced against the actual source code of `karlrobertc-art/BoardTrack` and `grp07-campussafe`. No AI suggestions were accepted without hands-on testing on disposable databases.
- **Ethical Compliance:** No credentials, production secrets, or private passwords were submitted or stored.